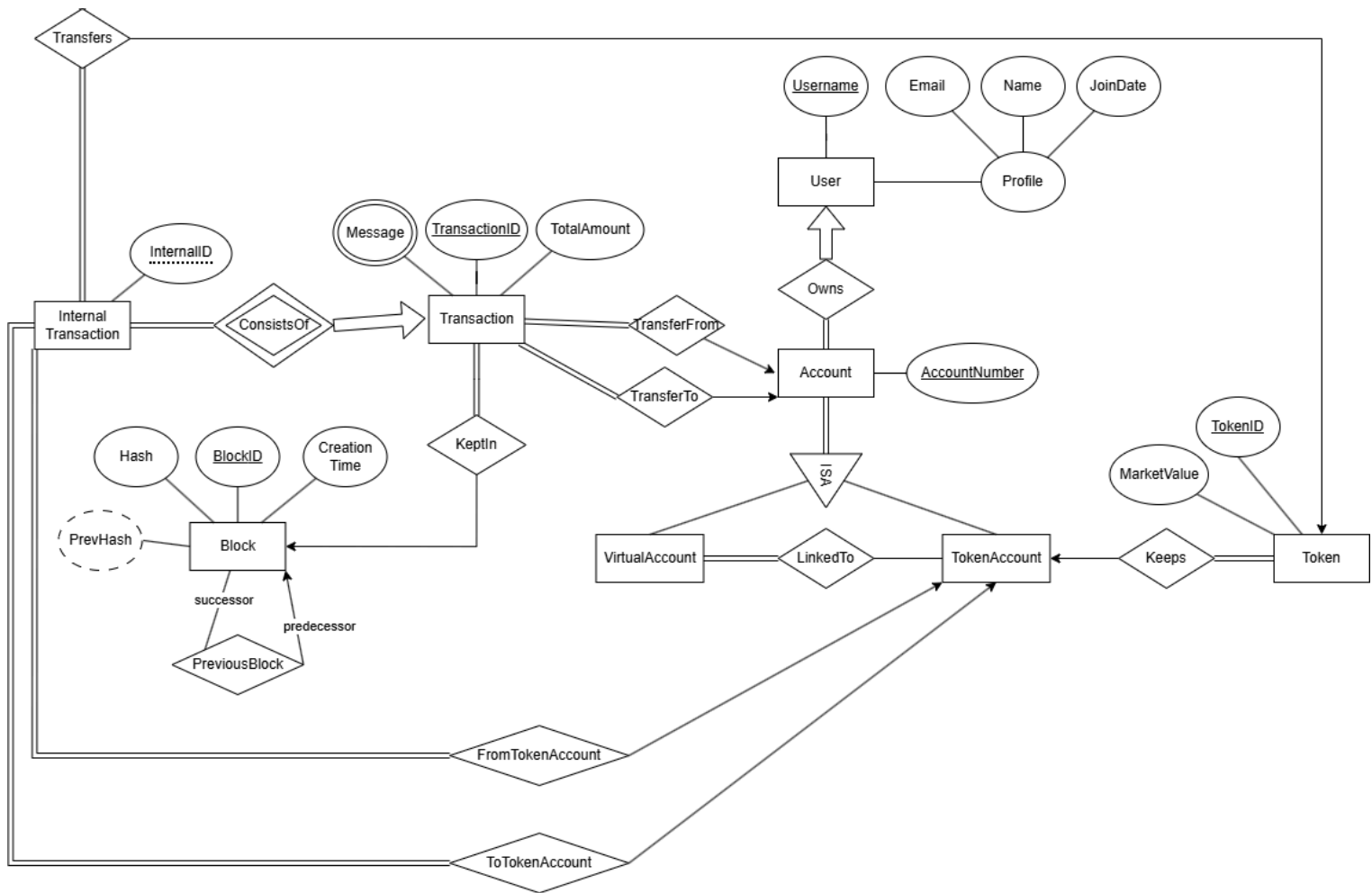


Q1

(a) ER diagram for the given scenario:



1. The specialization of Account into Token and Virtual is total and disjoint.
2. An account may exist without participating in any transaction.
3. A token may be transferred multiple times.
4. A block may contain zero transactions.

(b) Relational schema for the ER diagram:

User(Username, Name, Email, JoinDate)

—

Account(AccountNumber, Username)

FK1: Username references **User**(Username)

—

TokenAccount(AccountNumber)

FK1: AccountNumber references **Account**(AccountNumber)

—

VirtualAccount(AccountNumber)

FK1: AccountNumber references **Account**(AccountNumber)

—

LinkedTo(VirtualAccountNumber, TokenAccountNumber)

FK1: VirtualAccountNumber references **VirtualAccount**(AccountNumber)

FK2: TokenAccountNumber references **TokenAccount**(AccountNumber)

—
Token(TokenID, MarketValue, AccountNumber)

FK1: AccountNumber references **TokenAccount**(AccountNumber)

—
Block(BlockID, Hash, PrevHash, CreationTime, PrevBlockID)

FK1: PrevBlockID references **Block**(BlockID)

—
Transaction(TransactionID, TotalAmount, FromAccountNumber, ToAccountNumber, BlockID)

FK1: FromAccountNumber references **Account**(AccountNumber)

FK2: ToAccountNumber references **Account**(AccountNumber)

FK3: BlockID references **Block**(BlockID)

—
TransactionMessage(TransactionID, Message)

FK1: TransactionID references **Transaction**(TransactionID)

—
InternalTransaction(InternalID, TransactionID, TokenID, FromTokenAccountNumber, ToTokenAccountNumber)

FK1: TransactionID references **Transaction**(TransactionID)

FK2: TokenID references **Token**(TokenID)

FK3: FromTokenAccountNumber references **TokenAccount**(AccountNumber)

FK4: ToTokenAccountNumber references **TokenAccount**(AccountNumber)

Q2

(1)

-- List all available products in the system ordered by price.

```
SELECT
    product_id,
    description,
    price
FROM
    Product
ORDER BY
    price ASC;
```

product_id	description	price
1	4R photo	1.20
2	5R photo	2.30
5	2R photo card	2.50
3	8R photo	5.00
4	12R photo	15.00
6	Photo mug	230.00

(2)

-- Find the user IDs and names of all customers.

```
SELECT
    au.user_id,
    au.name
FROM
    Customer c
    JOIN AppUser au ON c.user_id = au.user_id;
```

user_id	name
1	Gebhard Nasato
2	Sunitha Aslan
5	Marcos Lannon
7	Fearghas Hamm
8	Rati Boon
10	Saoirse Meyer
11	Sievert Leclair
14	Adsila Fields
15	Lucija Romero

(3)

-- Find the customer name, phone numbers, and order ID of all orders that has "New" in the status.
-- If the customer has multiple phone numbers, concatenate them by commas in the result.
-- (Hint: use GROUP_CONCAT)

```
SELECT
    au.name,
    GROUP_CONCAT(cp.phone) AS phone,
    po.order_id
FROM
    PrintOrder po
    JOIN AppUser au ON po.user_id = au.user_id
    LEFT JOIN CustomerPhone cp ON po.user_id = cp.user_id
WHERE
    po.status LIKE '%New%'
GROUP BY
    po.order_id,
    au.name;
```

name	phone	order_id
Lucija Romero	246-6855,497-5406	4
Marcos Lannon	712-1926	5
Marcos Lannon	712-1926	6
Saoirse Meyer	529-5867	7

(4)

-- List all items of order ID 3, showing the item ID, quantity, product description, and the subtotal.
-- Subtotal equals to the product price times quantity.

```
SELECT
    i.item_id,
    i.quantity,
    p.description,
    (i.quantity * p.price) AS subtotal
FROM
    Item i
    JOIN Product p ON i.product_id = p.product_id
WHERE
    i.order_id = 3;
```

item_id	quantity	description	subtotal
1	5	4R photo	6.00
2	1	4R photo	1.20
3	2	Photo mug	460.00

(5)

-- List all orders by customer with user ID 5, showing the order ID, status, number of items (sum of quantity), and the total amount.

```
SELECT
    po.order_id,
    po.status,
    SUM(i.quantity) AS num_items,
    SUM(i.quantity * p.price) AS total_amount
FROM
    PrintOrder po
    LEFT JOIN Item i ON po.order_id = i.order_id
    LEFT JOIN Product p ON i.product_id = p.product_id
WHERE
    po.user_id = 5
GROUP BY
    po.order_id,
    po.status;
```

order_id	status	num_items	total_amount
3	Payment pending	8	467.20
5	New order	8	59.40
6	New re-order	8	59.40

(6)

-- Find the names of all users that is a customer and an admin user.

```
SELECT
    au.user_id,
    au.name
FROM
    Customer c
    JOIN Admin a ON c.user_id = a.user_id
    JOIN AppUser au ON c.user_id = au.user_id;
```

user_id	name
1	Gebhard Nasato
2	Sunitha Aslan
14	Adsila Fields

(7)

-- Find all admin users is not a supervisor.

```
SELECT
    a.user_id,
    au.name
FROM
    Admin a
JOIN AppUser au ON a.user_id = au.user_id
WHERE
    a.user_id NOT IN (
        SELECT
            DISTINCT supervisor_id
        FROM
            Admin
        WHERE
            supervisor_id IS NOT NULL
    );
```

user_id	name
3	Kemuel Sala
4	Harley Shine
9	Dylan Bellomi
13	Inzhu McCoy
14	Adsila Fields

(8)

-- List all albums owned by admin users, in a descending order of the number of images in the album.
-- If there exist two albums with the same number of images, order them in descending order of the creation date.

```
SELECT
    al.*,
    COUNT(img.image_id) AS num_images
FROM
    Album al
JOIN Admin ad ON al.user_id = ad.user_id
LEFT JOIN Image img ON al.album_id = img.album_id
GROUP BY
    al.album_id,
    al.name,
    al.creation_date,
    al.user_id
ORDER BY
    num_images DESC,
    al.creation_date DESC;
```

album_id	name	creation_date	user_id	num_images
6	Scenic photos	2020-02-01	13	5
1	Stock photos	2019-11-21	1	5
2	Family reunion	2019-11-25	3	4

(9)

-- Find the number of times each of the images in album ID 1 is included in an order.
-- An image that is placed in the same order multiple times should be counted as once only.
-- Images that has never been ordered must also be shown.

```
SELECT
    img.*,
    COUNT(DISTINCT i.order_id) AS order_count
FROM
    Image img
    LEFT JOIN Item i ON img.album_id = i.album_id
    AND img.image_id = i.image_id
WHERE
    img.album_id = 1
GROUP BY
    img.album_id,
    img.image_id,
    img.filename;
```

album_id	image_id	filename	order_count
1	1	IMG_1204.jpg	1
1	2	IMG_1206.jpg	3
1	3	IMG_1207.jpg	1
1	4	IMG_1211.jpg	1
1	5	IMG_1215.jpg	0

(10)

-- Find all customer name of orders that has the greatest number of order items (not considering quantity).

```
SELECT
    DISTINCT au.name
FROM
    PrintOrder po
    JOIN Item i ON po.order_id = i.order_id
    JOIN AppUser au ON po.user_id = au.user_id
GROUP BY
    po.order_id,
    au.user_id,
    au.name
HAVING
    COUNT(i.item_id) = (
        SELECT
            MAX(item_count)
        FROM
            (
                SELECT
                    COUNT(item_id) AS item_count
                FROM
                    Item
                GROUP BY
                    order_id
            ) AS counts
    );
```

name
Marcos Lannon
Sunitha Aslan